Code Quality and Refactoring

Module 3 — Onboarding & Industrialize the Codebase

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

We want to fix the problems you found in `triage.py`, one at a time. For each one we will see:

- the **concept**: what the problem is called, and why it hurts
- the **fix**: I will write and commit them in the course repository live (they are also shown on the slides as a diff). You will make the same changes in your clone, and commit/push them to your fork

First, a short setup: today you make your first commits, so git needs to know who you are and GitHub needs to know it is you.

Concepts covered: software quality, code smells, refactoring, best practices (KISS, conventions, no magic numbers, DRY, anticipation of change, separation of concerns), mutable default arguments, global state, context managers, error handling (try/except, LBYL and EAFP), anti-patterns.

Set up git for writing

Who you are: git identity

Every commit records its **author**: a name and an e-mail address. Git takes them from its configuration, so set them once, on every computer you use:

```
git config --global user.name  "Your Name"
git config --global user.email "you@studenti.unica.it"
git config --global --list        # check: user.name and user.email are there
```

`--global` saves the setting for all your repositories. Use the same e-mail address as your GitHub account (or add this one to your account), so GitHub links your commits to you.

Proving it is you: a personal access token

`git push` sends your commits to GitHub, and GitHub accepts them only from the owner of the fork. GitHub does **not** accept your account password on the command line: it wants a **personal access token**, a password that GitHub creates for you, with only the permissions you choose. To create a classic token:

- on GitHub: your photo → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**
- give it a name (for example `isde laptop`), an expiration date after the end of the course, and tick the scopes **`public_repo`** (push to your public fork) and **`workflow`** (needed for CI files)
- copy the token at once: GitHub shows it **only once**. Treat it like a password: never put it in a file of the repository, a slide or a chat

Then tell git to remember it, so that you type it only once, at your first `git push` (Step 0):

```
git config --global credential.helper store   # saves user name and token in ~/.git-credentials
# at the first git push:  Username: <your-user>   Password: paste the token (nothing is shown)
```

`store` keeps the token as plain text in your home folder: anyone who can read your files can use it. If the token leaks, delete it on the same GitHub page and create a new one.

Where you work: lab-01-start, in your fork

You work on the branch `lab-01-start` (default of your cloned repo), and you commit there. `git push` sends your commits to the same branch of **your fork** on GitHub; the course repository does not change.

- I will type the same fixes live, pushing each step to the branch **`lab-01-solution`** of the course repository, `unica-isde/heron`
- `lab-01-start` in the course repository stays as it is: it is the starting point of everybody
- so there are two ways to the same result: the commits you write in your fork, and my commits in `lab-01-solution`. If you miss a step, or a lecture, you take it from `lab-01-solution`

Software quality

What is software quality?

Code has two kinds of requirements:

- **functional requirements**: the code does what it must do, correctly (for `heron`: the right verdict for each email)
- **non-functional requirements**: *how* the code is, beyond what it does. The code is
  - understandable
  - maintainable
  - testable
  - extendable
  - reusable

`triage.py` mostly meets its functional requirements: it gives a verdict for each sample. It fails almost all the non-functional ones. This module is about them.

Maintainability

Code is read far more often than it is written. Most of its life is **maintenance**: fixing, changing and extending it, usually by someone who did not write it (including you, six months later).

- **maintainable** code can be changed quickly, without introducing new defects
- every shortcut taken when the code was written makes later changes slower and riskier. The total cost of these shortcuts is called **technical debt**: like a loan, it must be paid back later, with interest
- the problems you found in `triage.py` are its technical debt; today we pay back a large part of it

Code smells

A **code smell** is a feature of the source code that *suggests* a deeper problem in its design.

- a smell is not necessarily a bug: the code may work today, but it is hard to understand, test, change or reuse
- smells are easy to spot once you know their names, but their definition is partly subjective: it depends on the language and on the team
- common smells: long functions, long parameter lists, obscure names, duplicated code, magic numbers, dead code, complex conditions, comments that explain *what* the code does (the code should say that itself)

The smells you might have found (1)

These are the main problems of `triage.py`. Each row is fixed by one commit today or in the next lectures:

Problem	**Concept**
`results.txt` committed, one commit named `stuff`	repository hygiene
hard-coded folder `/home/lgarcia/...`	anticipation of change
unexplained numbers `+3`, `>= 5`	magic numbers
`flagged=[]` in the signature	mutable default argument
`scores` and `verdicts` at module level	global mutable state
`open(...).read()`, `out.close()`	resource handling, context managers

The smells you might have found (2)

These are the main problems of `triage.py`. Each row is fixed by one commit today or in the next lectures:

Problem	Concept
`except:` that hides every error	error handling
the brand check copied three times	DRY
`subj`, `scores` never really used	dead code
one function does everything	separation of concerns
regular expressions to read headers	don't reinvent the wheel
everything in one file	modularity

What to do when you find a smell

1. Identify **what** is wrong, and **where**.
2. Understand its **consequences**: wrong results, slower changes, harder teamwork?
3. Weigh the **benefit** of fixing it against the cost and the risk of the change. You do not always have to fix it.
4. Look up the usual fix: most smells have a well-known refactoring.
5. Apply it, in **small steps**, checking after each step that nothing broke.

Refactoring

Refactoring means changing the structure of the code **without changing what it does**.

- the goal is the non-functional requirements: clearer, simpler, smaller pieces
- it is done in **small steps**; after each step you check that the behaviour is unchanged. The best check is an automated test suite, written *before* refactoring (we'll see it later)
- a change that *does* change the behaviour (for example, a bug fix) is not a refactoring: keep the two in separate commits, so a reviewer knows which commits may change results
- IDEs automate common refactorings (rename, extract function, move): use them, they do not make typing mistakes

Our safety net: the six verdicts

`heron` has no tests yet. Our safety net today is the output on the six samples. We save it once, then compare after every step:

```
python3 triage.py data/samples | sort > /tmp/before.txt
# ... change the code ...
python3 triage.py data/samples | sort | diff /tmp/before.txt -   # no output = same verdicts
```

`sort` makes the order of the lines stable. Every step today keeps the same verdicts and scores: the steps marked `fix:` change behaviour only in situations the samples do not cover (a missing folder, a second call), and the `refactor:` steps change nothing at all.

Each step: change, check, commit, push

For every fix today, I'll write the change live; then you make **the same change** in your clone and record it:

```
# ... edit triage.py as shown ...
python3 triage.py data/samples | sort | diff /tmp/before.txt -   # no output = same verdicts
git diff                        # read your change: lines with - are removed, lines with + are added
git add triage.py               # choose the files for the next commit
git commit -m "refactor: name the magic numbers"   # the same message as my commit of the step
git push                        # send the commit to your fork
```

- one commit per concept, with a message that says what changed: your history becomes a readable list of today's fixes
- after each step, I will push the same commit, with the same message, to `lab-01-solution`

If you fall behind

During the lecture, do not stop to catch up: follow the explanation, and complete the missing steps after the lecture.

You find the solution of every step in the branch **`lab-01-solution`** of the course repository: open `unica-isde/heron` on GitHub, choose the branch `lab-01-solution`, and click the list of commits. Each commit is one step, with its message and its full change.

Step 0: repository hygiene

Before touching the code, clean the **repository** around it:

- a `.gitignore` file lists the files git must never track: generated output (`results.txt`), environments (`.venv/`), caches (`__pycache__/`), secrets (`.env`)
- a `.gitignore` does not affect files git already tracks, so `results.txt` is **untracked** explicitly: `git rm --cached results.txt` (the file stays on disk)
- two short documents: a **README** (what the project is, how to run it) and a **LICENSE** (who may reuse the code, and on which conditions).

Step 0, your turn (1/3): a license, on GitHub

Without a license, nobody may legally copy or change the code. `heron` uses the **MIT license**: anyone may reuse the code, if they keep the copyright notice. GitHub writes the file for you. On the GitHub page of **your fork**, on the branch `lab-01-start`:

1. click **Add file** → **Create new file**, and type the file name `LICENSE`
2. click the button **Choose a license template** that appears, choose **MIT License**, fill in the year and your full name, and click **Review and submit**
3. click **Commit changes...**, replace the proposed message with `chore: add the MIT license`, keep **Commit directly to the `lab-01-start` branch**, and confirm

The commit now exists in your fork on GitHub, but not yet in your clone: the slide after next downloads it.

Step 0, your turn (2/3): a .gitignore, on GitHub

In the same way, click **Add file → Create new file** and type the name `.gitignore`. GitHub then offers **Choose .gitignore template**: choose **Python**. The template already lists what every Python project ignores: caches (`__pycache__/`), environments (`.venv`), build output (`dist/`), tool caches, secrets (`.env`).

`heron` writes one more file that git must not track, its output. Add this line at the end of the file:

```
results.txt
```

Commit it with the message `chore: add a gitignore`, directly on `lab-01-start`.

Step 0, your turn (3/3): back in your clone

Download the two commits you made on GitHub, then untrack `results.txt`, write a short `README.md` in your own words (what `heron` does, and the command that runs it), and commit:

```
git pull                          # download the new commits of your fork: LICENSE and .gitignore appear
git rm --cached results.txt       # stop tracking it; the file stays on disk
git add README.md
git status                        # 2 changes: new README.md, results.txt deleted
git commit -m "chore: add a readme, stop tracking results.txt"
git push                          # first push: your user name, then the token as password
```

- `git pull` brings the commits of your fork on GitHub into your clone. Run it **before** you commit in the clone: otherwise the two histories split, and git asks you to merge them
- run `triage.py` again, then `git status`: `results.txt` changes at every run, but git no longer lists it

Best practices

KISS and coding conventions

KISS: *Keep It Simple, Stupid* (or *Keep It Short and Simple*). The simplest solution that works is the easiest to read, test and change.

> *Simple is better than complex. Complex is better than complicated. — The Zen of Python (`import this`)*

Follow the conventions of your language and team. For Python, **PEP 8** sets the layout and the names:

- `snake_case` for functions and variables, `UPPER_CASE` for constants, `CamelCase` for classes
- names that say what a thing **is**: `from_domain`, not `m1`; `reply_to`, not `rt`

You will not check PEP 8 by hand: a tool (ruff) does it for you.

Anticipation of change

Change is certain: requirements become clearer, users ask for new features, the environment changes (another machine, another folder). Good code **expects** change: values that may change are not buried in the code.

The default folder of `triage.py` exists only on its author's laptop. The fix, step 1:

```
-def check_mail(folder="/home/lgarcia/mail_export/", flagged=[]):
+def check_mail(folder, flagged=[]):
...
-         raw = open(folder + "/" + fn, encoding="utf-8", errors="ignore").read()
+         raw = open(os.path.join(folder, fn), encoding="utf-8", errors="ignore").read()
...
-     if len(sys.argv) > 1:
-         check_mail(sys.argv[1])
-     else:
-         check_mail()
+     if len(sys.argv) != 2:
+         print("usage: python triage.py <folder-with-eml-files>", file=sys.stderr)
+         sys.exit(2)
+     check_mail(sys.argv[1])
```

The folder is now a required argument, and `os.path.join` builds the path correctly on every operating system. A missing argument gives a clear message and exit code 2 (by convention: wrong usage).

Magic numbers

A **magic number** is a raw number in the code whose meaning is not stated. Why `+3` for a link? Why is `5` the phishing threshold? A reader cannot tell, and if the same value appears in several places, a change must find them all.

The fix, step 2: give each number a name, in one place at the top of the file.

```
+# Points added by each indicator, and the score thresholds of each verdict.
+W_KEYWORD = 1
+W_IP_LINK = 3
+W_PUNYCODE_LINK = 3
+W_BRAND_MISMATCH = 3
+W_AUTH_FAIL = 2
+W_REPLY_TO_MISMATCH = 2
+PHISHING_THRESHOLD = 5
+SUSPICIOUS_THRESHOLD = 3
...
-                 s = s + 3  # ip address url, very bad
+                 s = s + W_IP_LINK
...
-         if s >= 5:
+         if s >= PHISHING_THRESHOLD:
```

A refactoring: the verdicts do not move.

Python pitfalls

Mutable default arguments: the concept

Python evaluates a **default argument value once**, when the `def` statement runs, not at every call. If the default is a **mutable** object (a list, a dict, a set), every call that uses the default shares the **same** object, and changes to it survive between calls.

`check_mail(folder, flagged=[])` appends phishing emails to `flagged`. Call it twice in the same program:

```
>>> import triage
>>> triage.check_mail("data/samples")
...
flagged: ['password-reset-urgent.eml', 'prize-scam.eml', 'paypal-impersonation.eml']
>>> triage.check_mail("data/samples")
...
flagged: ['password-reset-urgent.eml', 'prize-scam.eml', 'paypal-impersonation.eml', 'password-reset-urgent.eml', 'prize-scam.eml', 'paypal-imperso
```

The second call reports six flagged emails out of six files: the list of the first call was still there.

Mutable default arguments: the fix

The standard fix: use `None` as the default, and create a new object **inside** the function, at every call (step 3):

```
-def check_mail(folder, flagged=[]):
+def check_mail(folder, flagged=None):
+    if flagged is None:
+        flagged = []
```

Immutable defaults (`None`, numbers, strings, tuples) are safe, because nobody can change them.

Global mutable state

`scores` and `verdicts` are **global variables**: they live at module level and every call of `check_mail()` appends to them. The same experiment as before shows the effect: after two calls, `triage.verdicts` holds **12** entries for 6 emails, and the second report prints every email twice.

Global mutable state couples every part of the program that reads or writes it: the result of a call depends on what earlier calls did, which makes the code hard to understand and to test.

The fix, step 4: local variables, and a **return value** instead of a shared list.

```
-scores = {}
-verdicts = []
...
 def check_mail(folder, flagged=None):
     if flagged is None:
         flagged = []
+    scores = {}
+    verdicts = []
...
     print("flagged:", flagged)
+    return verdicts
```

Releasing resources: context managers

An open file uses a resource of the operating system. `open(...).read()` never closes the file; `out.close()` is skipped if `write()` raises an exception. In a long-running program, leaked files pile up until the system refuses to open more.

A `with` block is a **context manager**: it closes the file when the block ends, **in every case**, also when an exception is raised. Step 5:

```
-         raw = open(os.path.join(folder, fn), encoding="utf-8", errors="ignore").read()
+         with open(os.path.join(folder, fn), encoding="utf-8", errors="ignore") as f:
+             raw = f.read()
...
-     out = open("results.txt", "w")
-     out.write(str(verdicts))
-     out.close()
+     with open("results.txt", "w", encoding="utf-8") as out:
+         out.write(str(verdicts))
```

The same pattern works for network connections, database connections and locks.

Error handling

Errors are expected

Code must decide what happens when something goes wrong: a missing header, a folder that does not exist, an input of the wrong type. There are two styles:

- **LBYL**, *Look Before You Leap*: check the condition first, then act (`if match is None: ...`)
- **EAFP**, *Easier to Ask Forgiveness than Permission*: just act, and handle the exception if it comes (`try: ... except FileNotFoundError: ...`)

Python supports both. The next slides show the syntax, then where `heron` uses each style.

try / except: the syntax

```
try:
    ...                         # code that may raise an exception
except ValueError:
    ...                         # runs only if a ValueError was raised
except (KeyError, IndexError):
    ...                         # one handler for several exception types
else:
    ...                         # runs only if the try block raised nothing
finally:
    ...                         # always runs last, exception or not: clean-up
```

- Python tries the `except` clauses **in order** and runs the first one that matches. An exception matches its own class and every parent class: `except Exception` catches almost everything.
- An exception that no clause matches continues up to the caller.

LBYL: check first

LBYL fits *expected* situations that are cheap to check. A missing header is normal in email: `re.search` then returns `None`, and calling `.group()` on `None` raises `AttributeError`.

`triage.py` hid that with a bare `except:`. The fix (step 6, part 1) checks for `None` explicitly, once, in a small function:

```
def header(raw, name):
    """Return the value of the first "name:" header line, or "?" if there is none."""
    match = re.search(name + ": (.*)", raw)
    if match is None:
        return "?"
    return match.group(1)
```

Drawback of LBYL: many checks make code long, and the world can change between the check and the action (another program deletes the file just after you checked that it exists).

EAFP: act, then handle

EAFP fits *exceptional* situations. A folder that does not exist is a user error, and `os.listdir` already raises `FileNotFoundError` for it. The entry point catches **exactly** that exception (step 6, part 2):

```
-    check_mail(sys.argv[1])
+    try:
+        check_mail(sys.argv[1])
+    except FileNotFoundError:
+        print("error: no such folder:", sys.argv[1], file=sys.stderr)
+        sys.exit(2)
```

EAFP keeps the main path readable and handles both known and unexpected cases, but raising and catching an exception is slower than an `if`: use it when the exception is really an exception.

The bare except

`except:` with no exception type catches **everything**: a missing header, but also a typo in a variable name (`NameError`), a bug in your code, even `Ctrl+C` (`KeyboardInterrupt`). The error disappears, and a bug looks exactly like a missing header.

```
-         try:
-             rt = re.search("Reply-To: (.*)", raw).group(1)
-             m1 = re.search("@([a-zA-Z0-9.-]+)", frm).group(1)
-             m2 = re.search("@([a-zA-Z0-9.-]+)", rt).group(1)
-             if m1 != m2:
-                 s = s + W_REPLY_TO_MISMATCH
-         except:
-             pass
+         from_domain = sender_domain(frm)
+         reply_domain = sender_domain(header(raw, "Reply-To"))
+         if from_domain and reply_domain and from_domain != reply_domain:
+             s = s + W_REPLY_TO_MISMATCH
```

Rules: catch the **narrowest** exception type you can handle; never `except: pass`; if you must catch broadly (`except Exception`), at least log the error. Note also the new names: `from_domain` says what `m1` did not.

More best practices

DRY: don't repeat yourself

DRY: every piece of knowledge has **one** place in the code. Two copies of the same code drift apart: a bug fixed in one copy stays in the other. Copying blocks of code to reuse them is an anti-pattern called **cut-and-paste programming**.

`triage.py` checks three brands with three copies of the same two lines. Step 7 turns the brands into **data** and checks them with **one** loop:

```
+BRANDS = ["paypal", "microsoft", "amazon"]
...
-         if "paypal" in frm.lower() and "paypal.com" not in frm.lower():
-             s = s + W_BRAND_MISMATCH
-         if "microsoft" in frm.lower() and "microsoft.com" not in frm.lower():
-             s = s + W_BRAND_MISMATCH
-         if "amazon" in frm.lower() and "amazon.com" not in frm.lower():
-             s = s + W_BRAND_MISMATCH
+         for brand in BRANDS:
+             if brand in frm.lower() and brand + ".com" not in frm.lower():
+                 s = s + W_BRAND_MISMATCH
```

Adding a fourth brand is now one word in a list, not two more copied lines.

Dead code

Dead code is code that runs but whose result is never used, or that never runs at all (including commented-out code). It is common when research code moves into production. It still has to be read, understood and maintained, and it hides the code that matters.

Step 8 removes it from `triage.py`:

- `subj` was read from every email and never used
- the `scores` dictionary was filled only to count the emails, which `len(verdicts)` already does
- the comment `# TODO: make this nicer at some point` had no owner and no plan: a TODO belongs in the issue tracker

Do not keep dead code "just in case": git keeps every old version for you.

Separation of concerns: the problem

Separation of concerns (SoC): divide a problem into smaller sub-problems, and give each one to a separate piece of code with a **single responsibility**.

`check_mail()` does everything: it lists the folder, reads the files, reads the headers, computes six indicators, chooses the verdict, prints, and writes `results.txt`. A **long function** with many responsibilities is a smell:

- you must read all of it to change any part of it
- you cannot test one indicator alone, or reuse the scoring without printing and writing a file
- if a verdict is wrong, you do not know which part is responsible

A rule of thumb: if you need "and" to describe what a function does, split it.

Separation of concerns (1/2): scoring apart from output

We split the long function in two steps. Step 9 separates the **scoring** from the **input and output**: `check_mail()` now returns **data** instead of printing, and two new functions take the other jobs:

```
def verdict_for(score):
    """Map a score to a verdict."""
    if score >= PHISHING_THRESHOLD:
        return "PHISHING"
    if score >= SUSPICIOUS_THRESHOLD:
        return "suspicious"
    return "ok"
...
        verdicts.append((fn, verdict_for(s), s))
    return verdicts


def report(verdicts):
    """Print the verdicts and save them to results.txt."""
    print("checked", len(verdicts), "mails")
    ...
    print("flagged:", [v[0] for v in verdicts if v[1] == "PHISHING"])
```

The `flagged` parameter is gone: the flagged emails are simply the verdicts equal to `"PHISHING"`. The entry point now calls `report(check_mail(sys.argv[1]))`.

Separation of concerns (2/2): one function per indicator

`check_mail()` still computes six indicators inline. Step 10 gives each indicator its own small function, and `score_email()` adds them up:

```
URL_RE = re.compile("https?://[^\\s\"'<>]+")
IP_URL_RE = re.compile("https?://[0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+")
...
def link_score(raw):
    """Points for links to a bare IP address or to a punycode domain."""
    s = 0
    for url in URL_RE.findall(raw):
        if IP_URL_RE.match(url):
            s = s + W_IP_LINK
        if "xn--" in url:
            s = s + W_PUNYCODE_LINK
    return s
...
def score_email(raw):
    """Sum the points of every indicator for one raw message."""
    frm = header(raw, "From")
    return (keyword_score(raw) + link_score(raw) + brand_score(frm)
            + auth_score(raw) + reply_to_score(frm, header(raw, "Reply-To")))
```

The other indicators (`keyword_score`, `brand_score`, `auth_score`, `reply_to_score`) follow the same shape, and the two URL patterns are compiled once, at the top. Inside the loop of `check_mail()`, all the indicator lines become one: `s = score_email(raw)`.

Don't reinvent the wheel: the problem

An email is a **structured format** (RFC 5322 and MIME): a header can be folded over several lines, encoded, or written in many valid shapes. Regular expressions over the raw text are a home-made parser, and they misread real messages. Example: a sender name encoded in base64, which is legal and common:

```
From: =?utf-8?b?UGF5UGFsIFN1cHBvcnQ=?= <security@paypa1-secure.net>

regular expression sees:  =?utf-8?b?UGF5UGFsIFN1cHBvcnQ=?= <security@paypa1-secure.net>
email parser sees:        PayPal Support <security@paypa1-secure.net>
```

The regular expression never finds "paypal" in this sender, so the brand impersonation goes unnoticed. Attackers know such tricks.

Don't reinvent the wheel: the fix

If a good solution already exists, reuse it: well-known libraries are tested by many users, and your own version is unlikely to be better. Python's standard library has a complete email parser. Step 11:

```
+from email import message_from_string, policy
+from email.utils import parseaddr
...
-def sender_domain(address):
-    """Return the domain after the @ in an address, or None if there is no @."""
-    match = re.search("@([a-zA-Z0-9.-]+)", address)
-    if match is None:
+def sender_domain(header_value):
+    """Return the domain of the address in a From/Reply-To value, or None."""
+    _, address = parseaddr(header_value)
+    if "@" not in address:
         return None
...
-    frm = header(raw, "From")
+    msg = message_from_string(raw, policy=policy.default)
+    frm = str(msg.get("From", ""))
+    reply_to = str(msg.get("Reply-To", ""))
```

The home-made `header()` function is deleted. On the six samples the verdicts do not move: they contain no encoded headers.

Anti-patterns

Anti-patterns

A **design pattern** is a good, reusable solution to a common problem. An **anti-pattern** is the opposite: a common solution that looks reasonable but is ineffective and causes problems later.

Knowing their names helps you recognise them, in your code and in a review. The classic catalogue is *AntiPatterns* by Brown et al. (1998). `triage.py` contains several of them: the next slides name them.

Anti-patterns in triage.py

Anti-pattern	What it is	In `triage.py`	Fixed by
Spaghetti code	code with little or no structure	one long function, global variables	steps 9–10
Cut-and-paste programming	reuse by copying blocks	the three brand checks	step 7
Dead code	code that serves no purpose	`subj`, `scores`	step 8
Reinvent the wheel	rewrite what a library already does	regex email parser	step 11
Golden hammer	one familiar tool used for every problem	regular expressions for everything	step 11
Input kludge	user input handled with ad-hoc code that easily fails	raw regex on attacker-controlled emails	step 11

The remedy is always the same: refactoring, in small checked steps, plus a plan of the structure *before* writing production code.

Lock-in and the isolation layer

Reusing libraries brings two risks:

- **continuous obsolescence**: libraries change quickly, and a new version may no longer fit your code
- **vendor lock-in**: your code depends so much on one external component that every change in it forces changes everywhere

The remedy is an **isolation layer**: one part of your code is the only one that talks to the external component, and it offers the rest of the code a stable interface of your own. When the component changes, only the isolation layer changes.

For example, if only one function of your program calls an e-mail parsing library and returns your own data type, a new version of that library changes that one function, and nothing else.

Where we are

`triage.py` after step 11: the same verdicts, but every problem of today is fixed. Look at your history, and compare the start with the end:

```
git log --oneline                          # your 14 commits on top of "stuff"
git diff --stat <stuff-id> -- triage.py    # <stuff-id>: the ID of "stuff", the last line of git log
```

What is still wrong:

- everything is still in **one file**: we will split it into a package of modules
- it cannot be **installed**, and its dependencies are not declared
- nothing **checks** style, types or documentation automatically
- nothing **proves** it works: no tests

Takeaways

- your fork now holds one commit per fix on `lab-01-start`: git identity, a personal access token for `git push`, and the cycle change → check → `add` → `commit` → `push`; the complete solution is the branch `lab-01-solution`
- quality is more than correct results: maintainable code is understandable, testable and easy to change; the cost of shortcuts is **technical debt**
- a **code smell** suggests a design problem; **refactoring** removes it in small steps, without changing behaviour, checked after every step
- best practices: KISS, conventions (PEP 8), anticipation of change, no magic numbers, DRY, no dead code, separation of concerns, reuse libraries
- Python pitfalls: mutable default arguments, global mutable state, files not closed (use `with`)
- error handling: LBYL for expected cases, EAFP for exceptional ones; catch the narrowest exception, never a bare `except:`
- anti-patterns name common bad solutions: spaghetti code, cut-and-paste, dead code, reinvent the wheel, golden hammer, input kludge, lock-in

Appendix: every step in full

How to read the appendix

The slides of the lecture show only the key lines of each step. The next slides show the **complete change** of every step, as `git diff` prints it, so that you can check your code line by line, or type a step you missed.

- a line that starts with `-` is removed, a line that starts with `+` is added, a line that starts with a space is unchanged and only shows where the change goes
- `@@ -12,3 +12,3 @@` gives the position of the lines that follow: line 12 of the old file (3 lines shown) becomes line 12 of the new file
- when a step needs more than one slide, the change continues on the next slide, in order

Step 0 is not here: it is complete on its own slides.

Step 1 in full (1/2)

Commit `fix: take the mail folder as a command-line argument`: the complete change to `triage.py`.

```
@@ -12,3 +12,3 @@

-def check_mail(folder="/home/lgarcia/mail_export/", flagged=[]):
+def check_mail(folder, flagged=[]):
     files = os.listdir(folder)
@@ -17,3 +17,3 @@
             continue
-        raw = open(folder + "/" + fn, encoding="utf-8", errors="ignore").read()
+        raw = open(os.path.join(folder, fn), encoding="utf-8", errors="ignore").read()
         s = 0
@@ -73,5 +73,5 @@
 if __name__ == "__main__":
-    if len(sys.argv) > 1:
-        check_mail(sys.argv[1])
-    else:
-        check_mail()
+    if len(sys.argv) != 2:
+        print("usage: python triage.py <folder-with-eml-files>", file=sys.stderr)
+        sys.exit(2)
+    check_mail(sys.argv[1])
```

Step 1 in full (2/2)

Step 1, continued: the run command in `notes.txt` changes too. Add both files before the commit.

```
@@ -1,2 +1,2 @@
-run with: python triage.py (edit the folder path in the code!!)
+run with: python triage.py <folder>   (e.g. data/samples)
 score >=5 means phishing, >=3 suspicious. keywords list is in the code.
```

Step 2 in full (1/3)

Commit `refactor: name the magic numbers`: the complete change to `triage.py`.

```
@@ -9,2 +9,12 @@

+# Points added by each indicator, and the score thresholds of each verdict.
+W_KEYWORD = 1
+W_IP_LINK = 3
+W_PUNYCODE_LINK = 3
+W_BRAND_MISMATCH = 3
+W_AUTH_FAIL = 2
+W_REPLY_TO_MISMATCH = 2
+PHISHING_THRESHOLD = 5
+SUSPICIOUS_THRESHOLD = 3
+
 KEYWORDS = ["urgent", "verify", "suspended", "password", "expires", "act now",
@@ -30,3 +40,3 @@
             if kw in low:
-                s = s + 1
+                s = s + W_KEYWORD
         # links that look bad
```

Step 2 in full (2/3)

Step 2, continued: the change to `triage.py` goes on from the previous slide.

```
@@ -35,14 +45,14 @@
                if re.match("https?://[0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+", u):
-                   s = s + 3  # ip address url, very bad
+                   s = s + W_IP_LINK
                if "xn--" in u:
-                   s = s + 3
+                   s = s + W_PUNYCODE_LINK
            # sender says paypal/microsoft/amazon but domain is weird
            if "paypal" in frm.lower() and "paypal.com" not in frm.lower():
-               s = s + 3
+               s = s + W_BRAND_MISMATCH
            if "microsoft" in frm.lower() and "microsoft.com" not in frm.lower():
-               s = s + 3
+               s = s + W_BRAND_MISMATCH
            if "amazon" in frm.lower() and "amazon.com" not in frm.lower():
-               s = s + 3
+               s = s + W_BRAND_MISMATCH
            if "spf=fail" in low or "dmarc=fail" in low:
-               s = s + 2
+               s = s + W_AUTH_FAIL
            # reply-to different from from
@@ -53,3 +63,3 @@
                if m1 != m2:
-                   s = s + 2
+                   s = s + W_REPLY_TO_MISMATCH
            except:
```

Step 2 in full (3/3)

Step 2, continued: the change to `triage.py` goes on from the previous slide.

```
@@ -57,6 +67,6 @@
          scores[fn] = s
-         if s >= 5:
+         if s >= PHISHING_THRESHOLD:
              verdicts.append((fn, "PHISHING", s))
              flagged.append(fn)
-         elif s >= 3:
+         elif s >= SUSPICIOUS_THRESHOLD:
              verdicts.append((fn, "suspicious", s))
```

Step 3 in full

Commit `fix: remove the mutable default argument`: the complete change to `triage.py`.

```
@@ -22,3 +22,5 @@

-def check_mail(folder, flagged=[]):
+def check_mail(folder, flagged=None):
+    if flagged is None:
+        flagged = []
     files = os.listdir(folder)
```

Step 4 in full

Commit `fix: replace module-level state with local variables`: the complete change to `triage.py`.

```
@@ -6,5 +6,2 @@

-scores = {}
-verdicts = []
-
 # Points added by each indicator, and the score thresholds of each verdict.
@@ -25,2 +22,4 @@
         flagged = []
+    scores = {}
+    verdicts = []
     files = os.listdir(folder)
@@ -83,2 +82,3 @@
     print("flagged:", flagged)
+    return verdicts
```

Step 5 in full

Commit `fix: open files with context managers`: the complete change to `triage.py`.

```
@@ -28,3 +28,4 @@
             continue
-        raw = open(os.path.join(folder, fn), encoding="utf-8", errors="ignore").read()
+        with open(os.path.join(folder, fn), encoding="utf-8", errors="ignore") as f:
+            raw = f.read()
         s = 0
@@ -78,5 +79,4 @@
         print(" ", v[0], "->", v[1], "(score", str(v[2]) + ")")
-    out = open("results.txt", "w")
-    out.write(str(verdicts))
-    out.close()
+    with open("results.txt", "w", encoding="utf-8") as out:
+        out.write(str(verdicts))
     print("flagged:", flagged)
```

Step 6 in full (1/3)

Commit `fix: replace the bare excepts with explicit checks`: the complete change to `triage.py`.

```
@@ -19,2 +19,18 @@

+def header(raw, name):
+    """Return the value of the first "name:" header line, or "?" if there is none."""
+    match = re.search(name + ": (.*)", raw)
+    if match is None:
+        return "?"
+    return match.group(1)
+
+
+def sender_domain(address):
+    """Return the domain after the @ in an address, or None if there is no @."""
+    match = re.search("@([a-zA-Z0-9.-]+)", address)
+    if match is None:
+        return None
+    return match.group(1)
+
+
 def check_mail(folder, flagged=None):
```

Step 6 in full (2/3)

Step 6, continued: the change to `triage.py` goes on from the previous slide.

```
@@ -31,10 +47,4 @@
          s = 0
-         try:
-             frm = re.search("From: (.*)", raw).group(1)
-         except:
-             frm = "?"
-         try:
-             subj = re.search("Subject: (.*)", raw).group(1)
-         except:
-             subj = "?"
+         frm = header(raw, "From")
+         subj = header(raw, "Subject")
          low = raw.lower()
```

Step 6 in full (3/3)

Step 6, continued: the change to `triage.py` goes on from the previous slide.

```
@@ -60,10 +70,6 @@
          # reply-to different from from
-         try:
-             rt = re.search("Reply-To: (.*)", raw).group(1)
-             m1 = re.search("@([a-zA-Z0-9.-]+)", frm).group(1)
-             m2 = re.search("@([a-zA-Z0-9.-]+)", rt).group(1)
-             if m1 != m2:
-                 s = s + W_REPLY_TO_MISMATCH
-         except:
-             pass
+         from_domain = sender_domain(frm)
+         reply_domain = sender_domain(header(raw, "Reply-To"))
+         if from_domain and reply_domain and from_domain != reply_domain:
+             s = s + W_REPLY_TO_MISMATCH
          scores[fn] = s
@@ -88,2 +94,6 @@
          sys.exit(2)
-     check_mail(sys.argv[1])
+     try:
+         check_mail(sys.argv[1])
+     except FileNotFoundError:
+         print("error: no such folder:", sys.argv[1], file=sys.stderr)
+         sys.exit(2)
```

Step 7 in full

Commit `refactor: replace three copied brand checks with one loop (DRY)`: the complete change to `triage.py`.

```
@@ -16,2 +16,4 @@

+BRANDS = ["paypal", "microsoft", "amazon"]
+
 KEYWORDS = ["urgent", "verify", "suspended", "password", "expires", "act now",
@@ -60,9 +62,6 @@
                     s = s + W_PUNYCODE_LINK
-        # sender says paypal/microsoft/amazon but domain is weird
-        if "paypal" in frm.lower() and "paypal.com" not in frm.lower():
-            s = s + W_BRAND_MISMATCH
-        if "microsoft" in frm.lower() and "microsoft.com" not in frm.lower():
-            s = s + W_BRAND_MISMATCH
-        if "amazon" in frm.lower() and "amazon.com" not in frm.lower():
-            s = s + W_BRAND_MISMATCH
+        # sender name claims a brand but the address is not on the brand's domain
+        for brand in BRANDS:
+            if brand in frm.lower() and brand + ".com" not in frm.lower():
+                s = s + W_BRAND_MISMATCH
         if "spf=fail" in low or "dmarc=fail" in low:
```

Step 8 in full

Commit `refactor: remove dead code`: the complete change to `triage.py`.

```
@@ -1,3 +1,2 @@
 # quick phishing checker for the mail export - L. Garcia, march 2025
-# TODO: make this nicer at some point
 import re
@@ -40,3 +39,2 @@
         flagged = []
-    scores = {}
     verdicts = []
@@ -50,3 +48,2 @@
         frm = header(raw, "From")
-        subj = header(raw, "Subject")
         low = raw.lower()
@@ -73,3 +70,2 @@
             s = s + W_REPLY_TO_MISMATCH
-        scores[fn] = s
         if s >= PHISHING_THRESHOLD:
@@ -81,3 +77,3 @@
             verdicts.append((fn, "ok", s))
-    print("checked", len(scores), "mails")
+    print("checked", len(verdicts), "mails")
     for v in verdicts:
```

Step 9 in full (1/3)

Commit `refactor: separate scoring from input and output`: the complete change to `triage.py`.

```
@@ -36,8 +36,15 @@

-def check_mail(folder, flagged=None):
-    if flagged is None:
-        flagged = []
+def verdict_for(score):
+    """Map a score to a verdict."""
+    if score >= PHISHING_THRESHOLD:
+        return "PHISHING"
+    if score >= SUSPICIOUS_THRESHOLD:
+        return "suspicious"
+    return "ok"
+
+
+def check_mail(folder):
+    """Score every .eml file in folder; return (file name, verdict, score) tuples."""
     verdicts = []
-    files = os.listdir(folder)
-    for fn in files:
+    for fn in os.listdir(folder):
         if not fn.endswith(".eml"):
```

Step 9 in full (2/3)

Step 9, continued: the change to `triage.py` goes on from the previous slide.

```
@@ -70,9 +77,8 @@
                 s = s + W_REPLY_TO_MISMATCH
-            if s >= PHISHING_THRESHOLD:
-                verdicts.append((fn, "PHISHING", s))
-                flagged.append(fn)
-            elif s >= SUSPICIOUS_THRESHOLD:
-                verdicts.append((fn, "suspicious", s))
-            else:
-                verdicts.append((fn, "ok", s))
+            verdicts.append((fn, verdict_for(s), s))
+    return verdicts
+
+
+def report(verdicts):
+    """Print the verdicts and save them to results.txt."""
     print("checked", len(verdicts), "mails")
@@ -82,4 +88,4 @@
             out.write(str(verdicts))
-    print("flagged:", flagged)
-    return verdicts
+    print("flagged:", [v[0] for v in verdicts if v[1] == "PHISHING"])
+
```

Step 9 in full (3/3)

Step 9, continued: the change to `triage.py` goes on from the previous slide.

```
@@ -90,3 +96,3 @@
      try:
-         check_mail(sys.argv[1])
+         report(check_mail(sys.argv[1]))
      except FileNotFoundError:
```

Step 10 in full (1/4)

Commit `refactor: one function per indicator`: the complete change to `triage.py`.

```
@@ -20,2 +20,6 @@

+URL_RE = re.compile("https?://[^\\s\"'<>]+")
+IP_URL_RE = re.compile("https?://[0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+")
+
+
 def header(raw, name):
@@ -36,2 +40,50 @@

+def keyword_score(raw):
+    """One point for each pressure keyword found anywhere in the message."""
+    low = raw.lower()
+    return sum(W_KEYWORD for kw in KEYWORDS if kw in low)
+
+
+def link_score(raw):
+    """Points for links to a bare IP address or to a punycode domain."""
+    s = 0
+    for url in URL_RE.findall(raw):
+        if IP_URL_RE.match(url):
+            s = s + W_IP_LINK
+        if "xn--" in url:
+            s = s + W_PUNYCODE_LINK
+    return s
+
+
```

Step 10 in full (2/4)

Step 10, continued: the change to `triage.py` goes on from the previous slide.

```
+def brand_score(frm):
+    """Points when the sender name claims a brand but the address is elsewhere."""
+    low = frm.lower()
+    return sum(W_BRAND_MISMATCH for brand in BRANDS
+               if brand in low and brand + ".com" not in low)
+
+
+def auth_score(raw):
+    """Points when SPF or DMARC failed."""
+    low = raw.lower()
+    if "spf=fail" in low or "dmarc=fail" in low:
+        return W_AUTH_FAIL
+    return 0
+
+
+def reply_to_score(frm, reply_to):
+    """Points when replies go to a different domain than the sender's."""
+    from_domain = sender_domain(frm)
+    reply_domain = sender_domain(reply_to)
+    if from_domain and reply_domain and from_domain != reply_domain:
+        return W_REPLY_TO_MISMATCH
+    return 0
+
+
```

Step 10 in full (3/4)

Step 10, continued: the change to `triage.py` goes on from the previous slide.

```
+def score_email(raw):
+    """Sum the points of every indicator for one raw message."""
+    frm = header(raw, "From")
+    return (keyword_score(raw) + link_score(raw) + brand_score(frm)
+            + auth_score(raw) + reply_to_score(frm, header(raw, "Reply-To")))
+
+
 def verdict_for(score):
@@ -53,26 +105,3 @@
             raw = f.read()
-        s = 0
-        frm = header(raw, "From")
-        low = raw.lower()
-        for kw in KEYWORDS:
-            if kw in low:
-                s = s + W_KEYWORD
-        # links that look bad
-        urls = re.findall("https?://[^\\s\"'<>]+", raw)
-        for u in urls:
-            if re.match("https?://[0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+", u):
-                s = s + W_IP_LINK
-            if "xn--" in u:
-                s = s + W_PUNYCODE_LINK
-        # sender name claims a brand but the address is not on the brand's domain
-        for brand in BRANDS:
-            if brand in frm.lower() and brand + ".com" not in frm.lower():
-                s = s + W_BRAND_MISMATCH
```

Step 10 in full (4/4)

Step 10, continued: the change to `triage.py` goes on from the previous slide.

```
-         if "spf=fail" in low or "dmarc=fail" in low:
-             s = s + W_AUTH_FAIL
-         # reply-to different from from
-         from_domain = sender_domain(frm)
-         reply_domain = sender_domain(header(raw, "Reply-To"))
-         if from_domain and reply_domain and from_domain != reply_domain:
-             s = s + W_REPLY_TO_MISMATCH
+         s = score_email(raw)
          verdicts.append((fn, verdict_for(s), s))
```

Step 11 in full (1/2)

Commit `refactor: read headers with the standard library email parser`: the complete change to `triage.py`.

```
@@ -1,5 +1,7 @@
 # quick phishing checker for the mail export - L. Garcia, march 2025
-import re
 import os
+import re
 import sys
+from email import message_from_string, policy
+from email.utils import parseaddr

@@ -24,16 +26,8 @@

-def header(raw, name):
-    """Return the value of the first "name:" header line, or "?" if there is none."""
-    match = re.search(name + ": (.*)", raw)
-    if match is None:
-        return "?"
-    return match.group(1)
-
-
```

Step 11 in full (2/2)

Step 11, continued: the change to `triage.py` goes on from the previous slide.

```
-def sender_domain(address):
-    """Return the domain after the @ in an address, or None if there is no @."""
-    match = re.search("@([a-zA-Z0-9.-]+)", address)
-    if match is None:
+def sender_domain(header_value):
+    """Return the domain of the address in a From/Reply-To value, or None."""
+    _, address = parseaddr(header_value)
+    if "@" not in address:
         return None
-    return match.group(1)
+    return address.rsplit("@", 1)[1].lower()

@@ -83,5 +77,7 @@
     """Sum the points of every indicator for one raw message."""
-    frm = header(raw, "From")
+    msg = message_from_string(raw, policy=policy.default)
+    frm = str(msg.get("From", ""))
+    reply_to = str(msg.get("Reply-To", ""))
     return (keyword_score(raw) + link_score(raw) + brand_score(frm)
-            + auth_score(raw) + reply_to_score(frm, header(raw, "Reply-To")))
+            + auth_score(raw) + reply_to_score(frm, reply_to))
```